

Reviewer Pack — Hadamard-Code Distance Defect Predicts NW-PRG Distinguisher ...

Entry 1a5be3baec68 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 10:22:28 UTC

Hadamard-Code Distance Defect Predicts NW-PRG Distinguisher Advantage

Entry ID: 1a5be3baec68 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 10:22:28 UTC

1. Conjecture

Field A (mathematical branch): Coding theory of binary Reed-Muller / Hadamard codes (minimum-distance defect of the design's combinatorial incidence code) **Field B** (complexity object): Nisan-Wigderson pseudorandom generator output bias against small parity-AND distinguishers

Statement:

Let D be an (m, l, a) -NW design on $n = |S_i|$ sets and let M_D in $\{0, 1\}^{n \times m}$ be its incidence matrix; let $d(D)$ be the minimum Hamming distance between any two distinct rows of M_D and let $d^* = 1 - a$ be the design's pairwise-overlap upper bound on agreement. For every hard predicate $f: \{0, 1\}^l \rightarrow \{0, 1\}$ of correlation ϵ with degree-1 $\text{GF}(2)$ polynomials, the maximum distinguishing advantage of any AND-of- c -parities distinguisher against $\text{NW}_{\{D, f\}}$ satisfies $\text{adv}(\text{NW}_{\{D, f\}}) \leq c * \epsilon * 2^{\{-(d(D) - 2(l - d^*)) / 2\}}$, and this bound is tight up to a constant for at least 30% of random hard f at every $(m, l, a, n \leq 20)$.

Rationale (proposer's reasoning):

NW analysis classically pays a $2^{\{ca\}}$ factor for c -parity distinguishers via the design's overlap a ; recasting M_D as a binary code lets the actual* row-distance $d(D)$ — typically much larger than the worst-case $2(l - a)$ bound — control distinguisher leakage through a Plotkin-style inner-product argument. Coding-theoretic distance has been almost entirely absent from NW PRG analyses (which use overlap, not minimum distance), and the gap $d(D) - 2(l - a)$ is exactly the slack that explains why concrete designs beat their generic guarantee. The conjecture is falsifier-friendly: any single (D, f, c) tuple where the empirical advantage exceeds the RHS by more than a small constant kills it.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bfd373d0560cbe6e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all enumerated ($m, l, a, n \leq 20$) configurations with l in $\{3, 4, 5\}$, m in $\{4..8\}$, c in $\{1, 2, 3\}$, and ≥ 30 random hard predicates f per design, the bound $\text{adv}(\text{NW}_{\{D, f\}}) \leq \text{ceps2}^{\{-(d(D)-2(l-a))/2\}}$ must hold for $\geq 99\%$ of (D, f, c) triples (upper-bound support), and the ratio adv/RHS must exceed 0.3 for $\geq 30\%$ of f in $\geq 80\%$ of designs (tightness).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.86	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Nisan-Wigderson generator combinatorial design Hadamard code minimum distance - NW pseudorandom generator distinguisher advantage parity AND correlation bound - incidence matrix combinatorial design Reed-Muller code distance pseudorandomness

Top relevant hits considered: - [<http://arxiv.org/abs/2106.11085v3>] On the Maximal Monotone Operators in Hadamard Spaces - [<http://arxiv.org/abs/1003.4003v1>] A Fourier-analytic Approach to Counting Partial Hadamard Matrices - [<http://arxiv.org/abs/1003.4001v1>] On the Asymptotic Existence of Hadamard Matrices - [<http://arxiv.org/abs/1311.6531v3>] Brains and pseudorandom generators - [<http://arxiv.org/abs/2410.08073v3>] Efficient Quantum Pseudorandomness from Hamiltonian Phase States - [<http://arxiv.org/abs/2510.04085v1>] Gluing Random Unitaries with Inverses and Applications to Strong Pseudorandom Unitaries - [<http://arxiv.org/abs/0909.3185v2>] Construction of Additive Reed-Muller Codes - [<http://arxiv.org/abs/1407.6185v4>] Relative generalized Hamming weights of q -ary Reed-Muller codes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import sys
from collections import defaultdict

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def walsh_hadamard_transform(f):
        n = len(f)
        T = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                T[i][j] = (-1)**(i & j)
        F = [0] * n
```

```

    for k in range(n):
        sum_val = 0
        for i in range(n):
            sum_val += f[i] * T[i][k]
        F[k] = sum_val / math.sqrt(n)
    return F

def max_correlation(f):
    F = walsh_hadamard_transform(f)
    eps = max(abs(x) for x in F)
    return eps

def generate_design(m, n):
    design = []
    while len(design) < m:
        candidate = [random.choice([0, 1]) for _ in range(n)]
        if all(candidate != row for row in design):
            design.append(candidate)
    return design

def min_hamming_distance(matrix):
    min_dist = float('inf')
    n = len(matrix[0])
    for i in range(len(matrix)):
        for j in range(i + 1, len(matrix)):
            dist = sum(1 for x, y in zip(matrix[i], matrix[j]) if x != y)
            min_dist = min(min_dist, dist)
    return min_dist

def and_of_c_parities(design, c):
    n = len(design[0])
    m = len(design)
    parities = []
    for comb in itertools.combinations(range(n), c):
        parity = [sum(design[i][j] for j in comb) % 2 for i in range(m)]
        parities.append(parity)
    return parities

def nw_distinction(design, predicate, parity):
    n = len(design[0])
    m = len(design)
    count_0 = 0
    count_1 = 0
    for i in range(2**n):
        seed = [int(x) for x in bin(i)[2:].zfill(n)]
        if predicate(seed) == parity[i]:
            count_1 += 1
        else:
            count_0 += 1
    return abs(count_0 - count_1)

l_values = [3, 4, 5]
m_values = range(4, 9)
c_values = [1, 2, 3]
n_max = 20

```

```

results = []
for l in l_values:
    a = l - 1
    d_star = l - a
    for m in m_values:
        design = generate_design(m, n_max)
        d_D = min_hamming_distance(design)
        for c in c_values:
            parities = and_of_c_parities(design, c)
            for parity in parities:
                eps = max_correlation(parity)
                RHS = c * eps * 2**(-(d_D - 2 * (l - d_star))) / 2)
                advantage = nw_distinction(design, lambda x: sum(x) % 2 == parity[0], parity)
                results.append({
                    "metric_name": "advantage",
                    "metric_value": advantage,
                    "instances_tested": 1,
                    "conjecture_holds": advantage <= RHS,
                    "counterexample": "" if advantage <= RHS else f"adv={advantage}, RHS={RHS}"
                })

mean_adv = sum(result["metric_value"] for result in results) / len(results)
std_adv = math.sqrt(sum((result["metric_value"] - mean_adv)**2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "seed": seed,
    "mean_advantage": mean_adv,
    "std_advantage": std_adv,
    "support_fraction": support_fraction
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_adv = sum(result["mean_advantage"] for result in results) / len(results)
    std_adv = math.sqrt(sum((result["mean_advantage"] - mean_adv)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.99) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_adv} std={std_adv} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["support_fraction"] < 0.99)
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c9b9fc0.py", line 120, in <module>
    result = run_trial(seed)
            ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c9b9fc0.py", line 95, in run_trial
    advantage = nw_distinction(design, lambda x: sum(x) % 2 == parity[0], parity)
               ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6c9b9fc0.py", line 72, in nw_distinction
    if predicate(seed) == parity[i]:
        ~~~~~^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError before producing any RESULT line, so neither the support nor the falsification conditions can be evaluated. Per rule 2, a crashed test yields INCONCLUSIVE. | next: Fix the nw_distinction harness so the parity vector is sized to match the number of distinguisher queries (length c), then re-run the enumeration over (m,l,a,n<=20) with >=5 seeds.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	24259
2	preregistration	claude_max	opus	0	0	7487
3	novelty	claude_max	opus	0	0	3106
4	novelty	claude_max	opus	0	0	8441
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14492
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13091
7	judge	claude_max	opus	0	0	5910

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 76787 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/1a5be3baec68.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1a5be3baec68.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1a5be3baec68.tar.gz` (if generated)